Experiment No 12

Experiment Title: MPI Program 4

Name : Rahul Bhati

Roll no : 2223416

Problem Statement:

MPI program to find sum of n integers in which processors are arranged in ring topology using MPI point-to-point blocking communication library calls.

Objectives

1. **Understand Ring Topology in MPI**: Learn how to simulate a ring communication pattern using MPI point-to-point communication.
2. **Implement Point-to-Point Communication**: Use MPI send and receive operations to pass data between processes arranged in a ring.
3. **Compute Sum Using Ring Communication**: Write an MPI program to find the sum of n integers where processes communicate in a ring topology.

Theory

MPI (Message Passing Interface) supports various communication patterns, including point-to-point communication, which allows processes to send and receive messages directly to and from other processes. A ring topology in MPI arranges processes in a circular manner, where each process communicates with its neighbors.

Key MPI Functions for Point-to-Point Communication:

1. **MPI_Init**: Initializes the MPI environment.
2. **MPI_Comm_size**: Determines the size of the communicator (number of processes).
3. **MPI_Comm_rank**: Determines the rank of the calling process within the communicator.
4. **MPI_Send**: Sends a message to a specified process.
5. **MPI_Recv**: Receives a message from a specified process.
6. **MPI_Finalize**: Terminates the MPI environment.

Conclusion

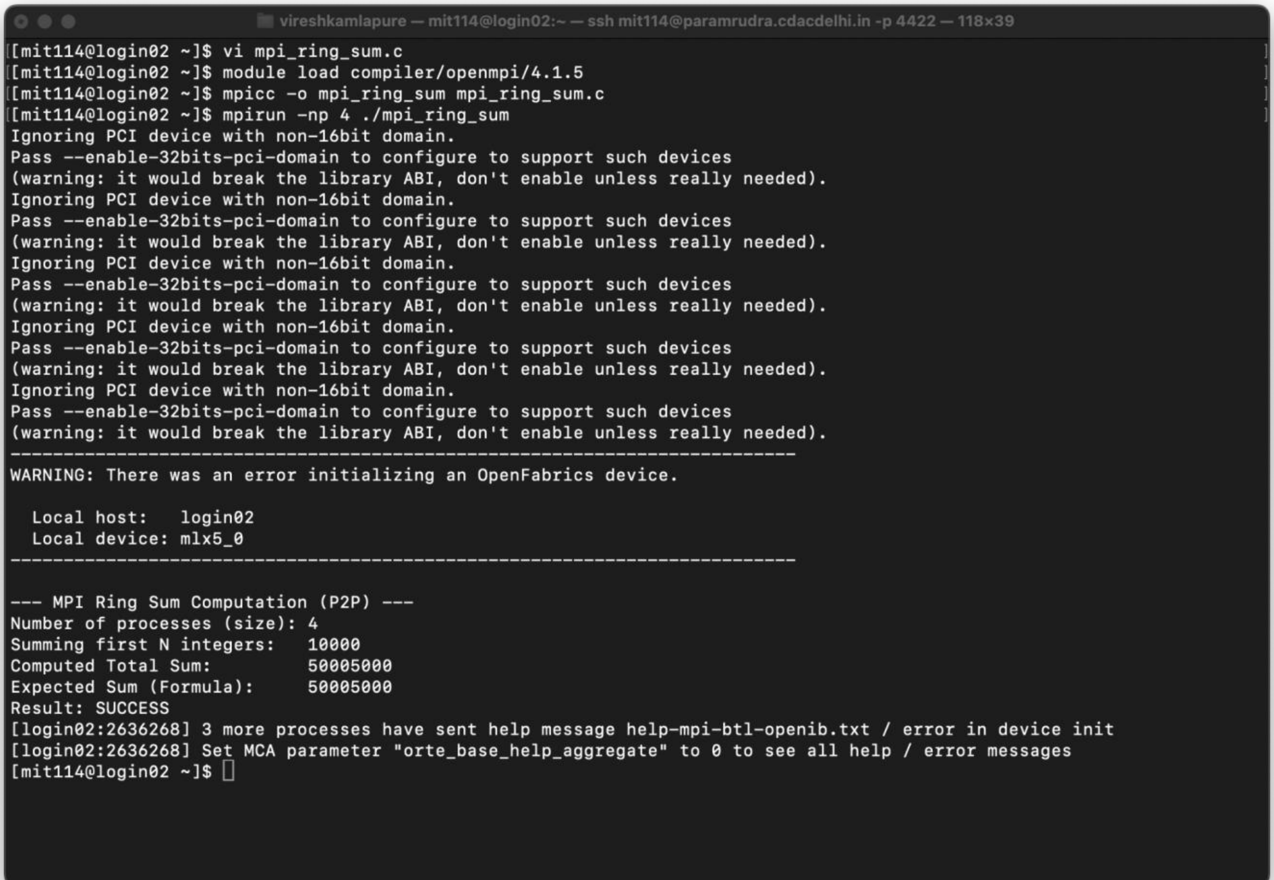
This MPI program demonstrates the use of point-to-point communication to compute the sum of n integers in a ring topology. Each process calculates its local sum and communicates with its neighbors to combine results until the total sum is computed by the root process. Understanding ring communication and point-to-point operations in MPI is essential for developing scalable and efficient parallel applications that utilize different communication patterns.

Source Code and Output /Screenshots:

```
#include <stdio.h>
```

```
#include <stdlib.h>  #include
<mpi.h>
int main(int argc, char* argv[]) {

   int rank, size;     const int
ROOT_RANK = 0;     long
long N = 10000;
   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);
   MPI_Status status;      if
(rank == ROOT_RANK) {
if (argc > 1) {
         N = atoll(argv[1]);
      }
      if (N <= 0) {
         fprintf(stderr, "Invalid N. Using default N=10000.\n");
         N = 10000;
      }
   }
   MPI_Bcast(&N, 1, MPI_LONG_LONG, ROOT_RANK, MPI_COMM_WORLD);
long long chunk_size = N / size;     long long start = (rank * chunk_size) + 1;
long long end = (rank + 1) * chunk_size;     if (rank == size - 1) {         end = N;
   }
   long long partial_sum = 0;     for
(long long i = start; i <= end; i++) {
partial_sum += i;
   }
   long long current_sum = partial_sum;
long long received_sum = 0;

   int dest_rank = (rank + 1) % size;     int
source_rank = (rank - 1 + size) % size;      if
(rank == ROOT_RANK) {
      MPI_Send(&current_sum, 1, MPI_LONG_LONG, dest_rank, 0, MPI_COMM_WORLD);

      MPI_Recv(&received_sum, 1, MPI_LONG_LONG, source_rank, 0, MPI_COMM_WORLD, &status);
current_sum = received_sum;

   } else {
      MPI_Recv(&received_sum, 1, MPI_LONG_LONG, source_rank, 0, MPI_COMM_WORLD, &status);
current_sum = received_sum + partial_sum;
      MPI_Send(&current_sum, 1, MPI_LONG_LONG, dest_rank, 0, MPI_COMM_WORLD);
   }


   if (rank == ROOT_RANK) {         printf("\n--- MPI Ring Sum
Computation (P2P) ---\n");         printf("Number of processes
(size): %d\n", size);         printf("Summing first N integers:
%lld\n", N);         printf("Computed Total Sum:         %lld\n",
current_sum);         long long expected_sum = (N * (N + 1)) / 2;
printf("Expected Sum (Formula):     %lld\n", expected_sum);
```

```
        if (current_sum == expected_sum) {
printf("Result: SUCCESS\n");
        } else {            printf("Result:
FAILED\n");
        }
    }
    MPI_Finalize();
return 0;
}
```



```
vireshkamlapure — mit114@login02:~ — ssh mit114@paramrudra.cdacdelhi.in -p 4422 — 118×39
[mit114@login02 ~]$ vi mpi_ring_sum.c
[mit114@login02 ~]$ module load compiler/openmpi/4.1.5
[mit114@login02 ~]$ mpicc -o mpi_ring_sum mpi_ring_sum.c
[mit114@login02 ~]$ mpirun -np 4 ./mpi_ring_sum
Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
--------------------------------------------------------------------------
WARNING: There was an error initializing an OpenFabrics device.

  Local host:   login02
  Local device: mlx5_0
--------------------------------------------------------------------------

--- MPI Ring Sum Computation (P2P) ---
Number of processes (size): 4
Summing first N integers:   10000
Computed Total Sum:         50005000
Expected Sum (Formula):     50005000
Result: SUCCESS
[login02:2636268] 3 more processes have sent help message help-mpi-btl-openib.txt / error in device init
[login02:2636268] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help / error messages
[mit114@login02 ~]$
```